



Exemplos de como consumir dados do Portal bigdata.pe.gov.br

Guia de Uso

Responsáveis:
Equipe da Plataforma de
Compartilhamento e Análise de Dados

Gerência de Projetos e Qualidade da Informação - DIC/ATI

Versão 1.7



Agência Estadual de
Tecnologia da Informação

Histórico de Alterações

Data	Versão	Descrição	Autor
13/11/2020	1.1	Início do documento	Eronita Van Leijden
30/12/2020	1.2	Incluindo seção Pentaho	Eronita Van Leijden
09/03/2021	1.3	Completando informação na seção Qlikview – incluindo informação sobre uso de filtro e paginação. Incluindo seção Insomnia	Eronita Van Leijden
01/10/2021	1.4	Incluindo seção Linha de Comando Linux e informação sobre chamado para CSATI qdo o consumo for por aplicação/sistema	Eronita Van Leijden
02/12/2021	1.5	Incluindo seção PowerBI e corrigindo URLs de todas as seções	Eronita Van Leijden
16/02/2024	1.6	Revisão geral após atualização. Adição de numeração das figuras Adição do tópico Python Alteração da logo da ATI	Matheus F. Silva
19/12/2024	1.7	Inclusão de seção R Studio e URLs com informações referente.	Igor Gomes

Sumário

1. Introdução.....	4
2. Tecnologia PENTAHO.....	4
4. Tecnologia Insomnia.....	8
5. Linha de Comando.....	10
6. Tecnologia Power Bi.....	10
7. Linguagem de programação Python.....	11
Configuração de ambiente.....	11
Consumindo um conjunto que não está na camada cache.....	13
Consumindo um conjunto que está na camada cache.....	16
8. R Studio.....	18

4. Incluir o componente **REST Client**
 - i. Configurar a aba **General** conforme imagem abaixo:

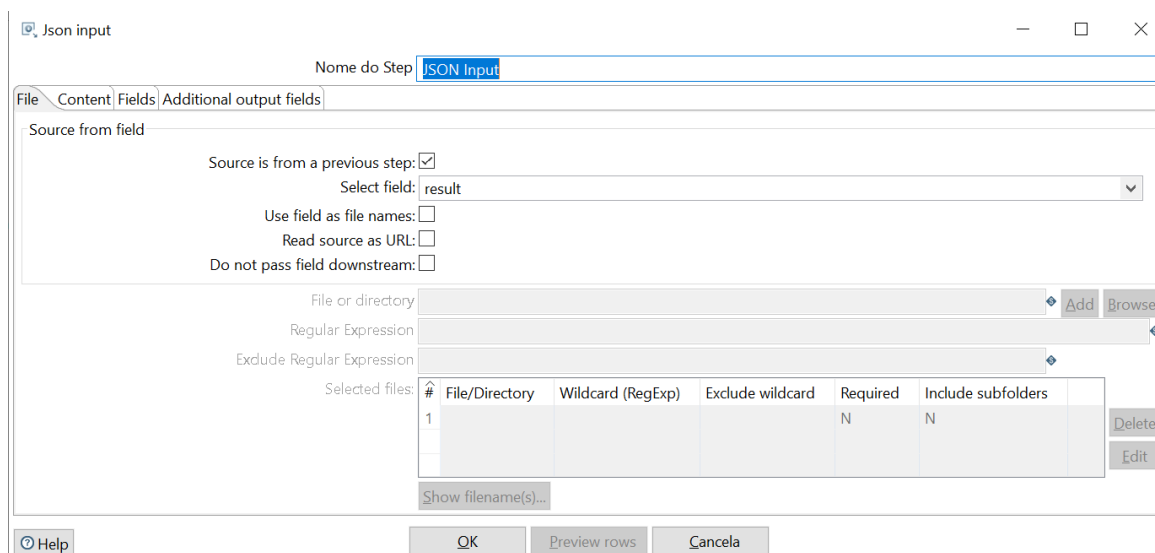


Figura 4

- ii. Configurar a aba **Fields**:
 - a. Na coluna **Name** inserir uma identificação do campo
 - b. Na coluna **Path** inserir o nome do(s) campo(s) exatamente igual como está na plataforma [bigdata.com.br](https://www.bigdata.com.br) com "\$.." na frente.

OBS: Para obter os campos disponíveis do conjunto de dados desejado pode-se consultar a informação do metadados disponível no detalhamento do conjunto de dados em <https://www.bigdata.pe.gov.br/catalogo-de-conjunto-de-dados-compartilhados>.

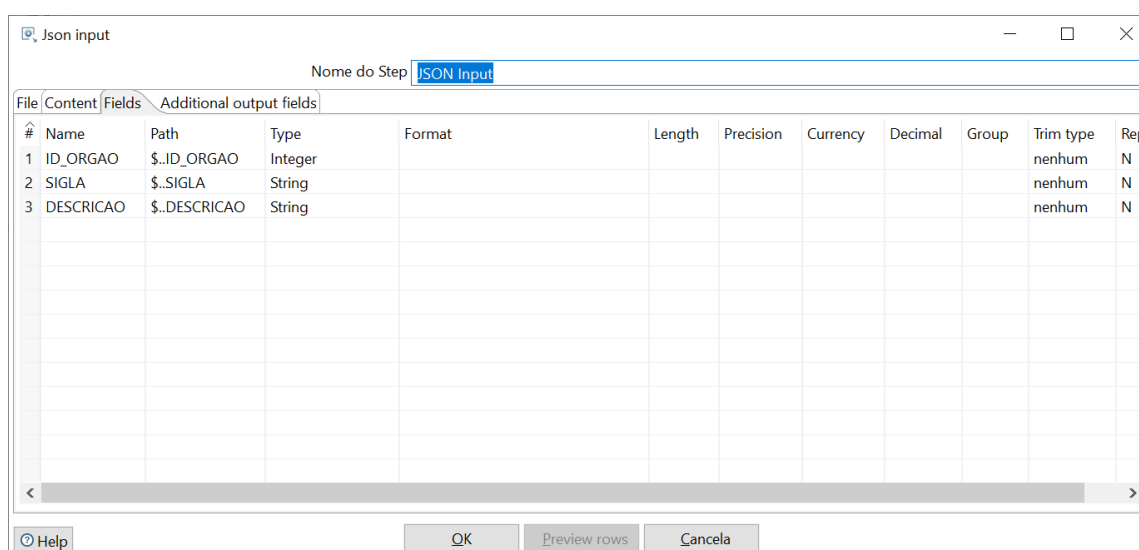


Figura 5

6. Incluir o componente **Text file output**

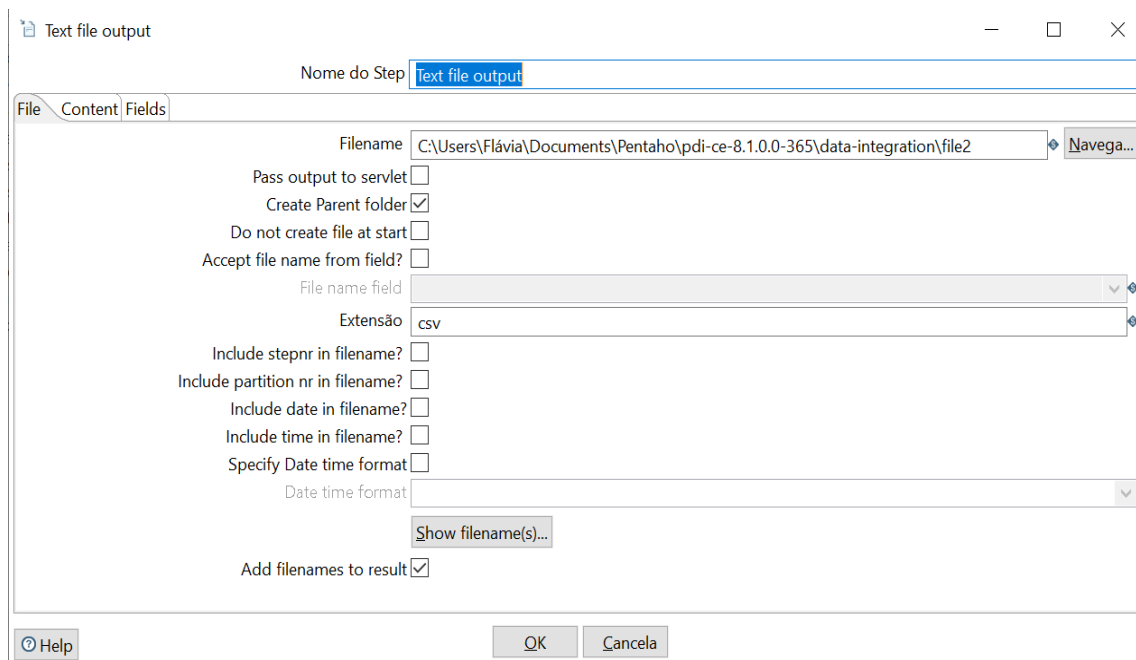


Figura 6

- i. Para configurar a aba **Fields** basta clicar no botão **Obtém campos**

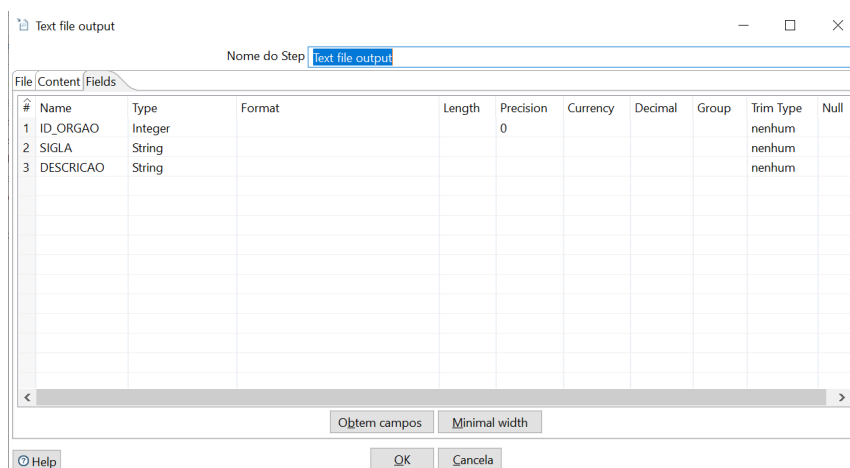


Figura 7

7. O resultado do fluxo criado deve ficar assim:

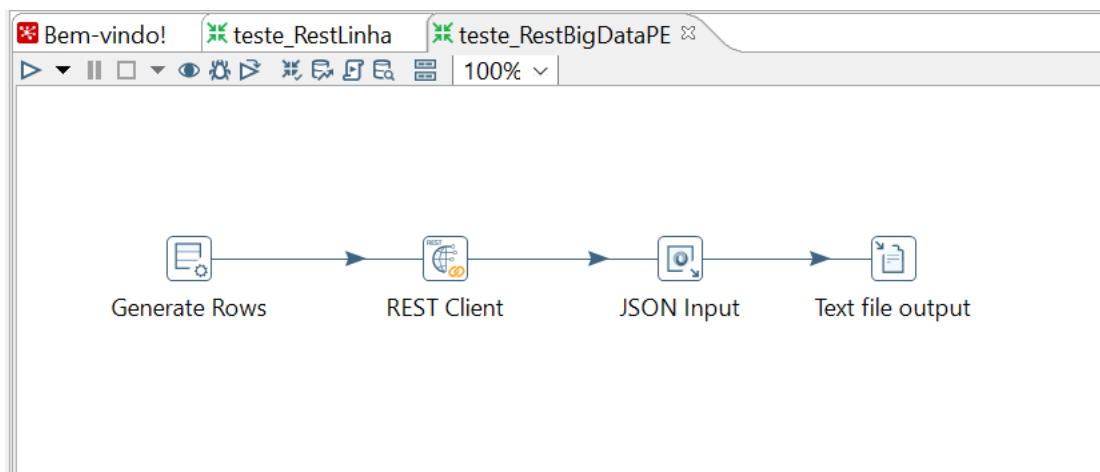


Figura 8

4. Tecnologia Insomnia

1. Instalação da ferramenta (<https://insomnia.rest/download>)
2. Caso não tenha login / não queira fazer login, basta clicar na opção **Use the local Scratch Pad**

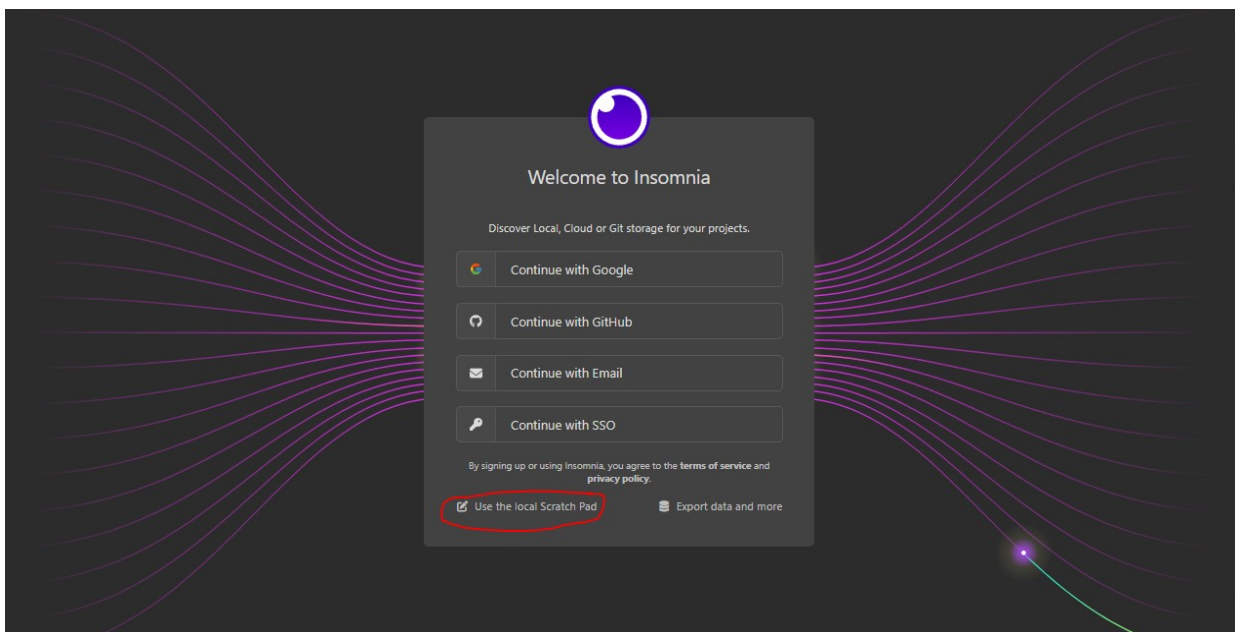


Figura 9

OBS: Essa será a tela que você verá após clicar em “Use the local Scratch Pad”

3. Para fazer requisição dos dados basta clicar no botão **New HTTP Request**
4. Agora clique na aba **Headers** e preencha os seguintes campos:

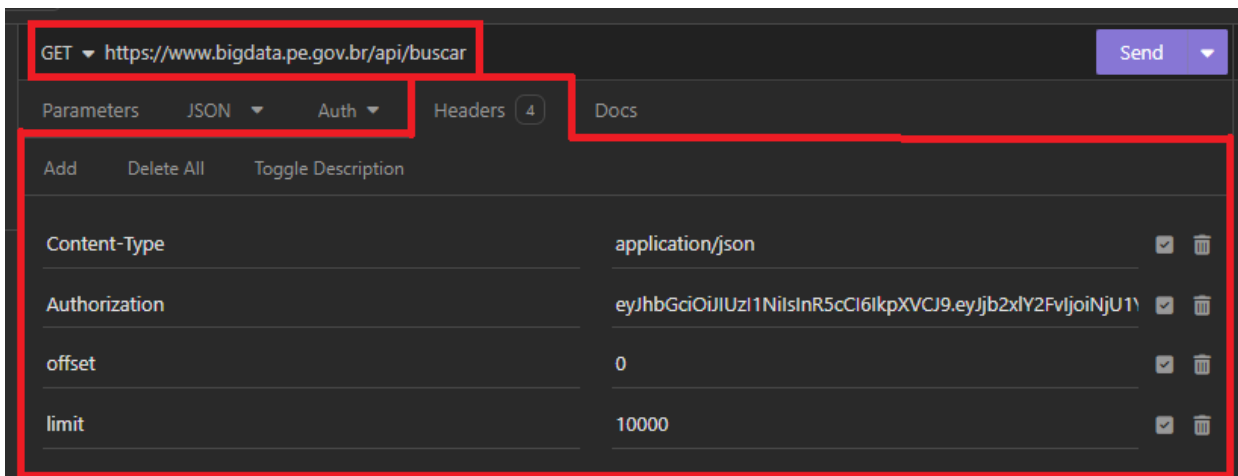


Figura 10

URL: <https://www.bigdata.pe.gov.br/api/buscar>

Header =>

Authorization = Valor dessa chave é o Token da collection do Big Data PE

Ex: e6ljVmnNjhlYTQ4MzFhOTM2MmNIMTYhMSlslmhdCI6...

Limit = Essa chave é o nº de registros que você quer trazer da collection.

Offset = A partir de qual registro a página irá começar.

5. Clique no botão **Send**.
6. **Ao lado direito da tela** será apresentado as informações retornadas.

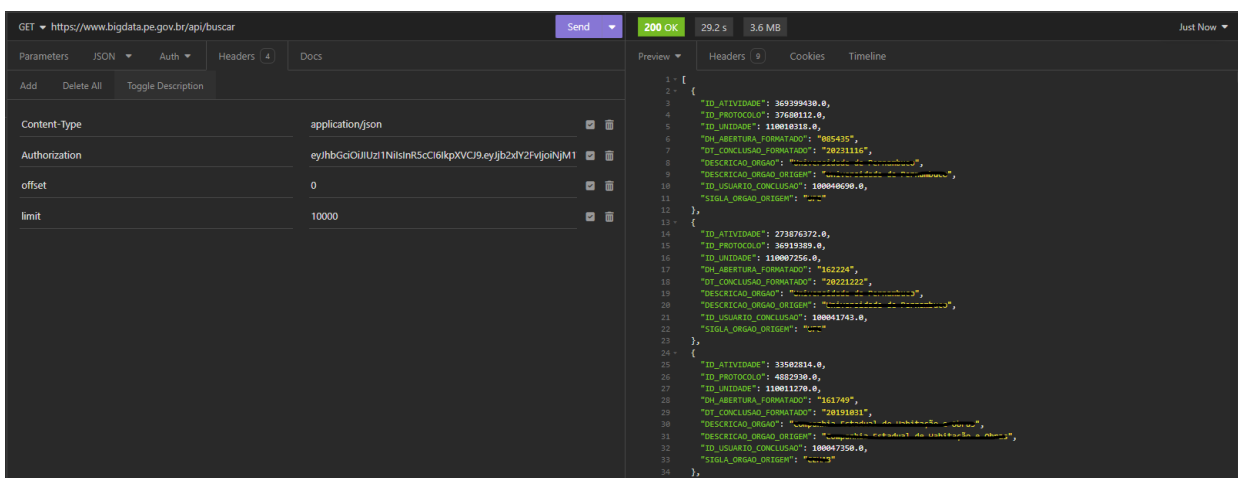


Figura 11

5. Linha de Comando

```
curl -H "Authorization: SeuTokenAqui" -o /caminho/para/sua/pasta/nome-do-arquivo-saida.extensao https://www.bigdata.pe.gov.br/api/buscar
```

O comando acima fica tudo em uma única linha

6. Tecnologia Power Bi

1. Abra o Power Bi
2. Clique em **obter dados** e selecione a opção **WEB**.

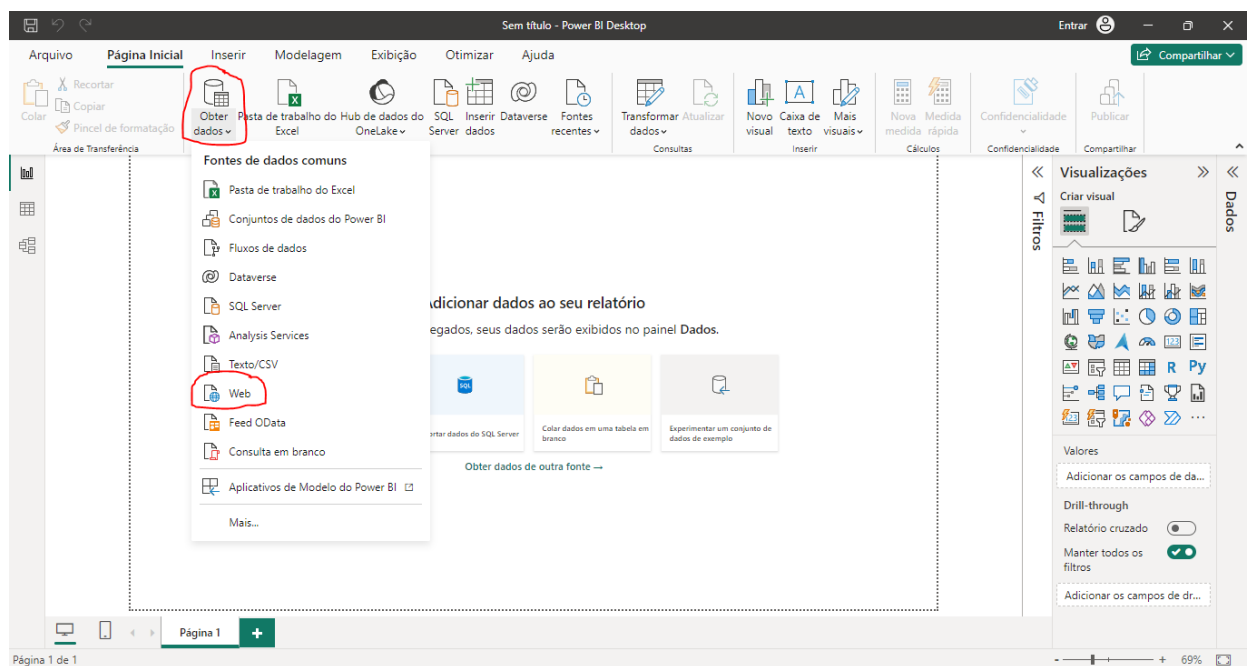


Figura 12

3. Você verá a tela abaixo, selecione a opção **Avançadas** e preencha as informações.

Authorization = Valor dessa chave é o **Token** do conjunto do Big Data PE

Ex: e6ljVmNjhiYTQ4MzFhOTM2MmNIMTYhMSIsImIhdCI6...

Da Web

☐ Básico ☒ Avançadas

Partes de URL ⓘ

https://www.bigdata.pe.gov.br/api/buscar

Adicionar parte

Visualização de URL

https://www.bigdata.pe.gov.br/api/buscar

Tempo limite do comando em minutos (opcional)

Parâmetros do cabeçalho de solicitação HTTP (opcional) ⓘ

Athorization dzJ5ttY4MIB9tw6gCMVLVu6SeYkt6iyk

Adicionar cabeçalho

OK Cancelar

Figura 13

Depois disso só clicar em OK.

7. Linguagem de programação Python

Configuração de ambiente

Caso você não tenha o python instalado na sua máquina, recomendo que faça download do **Anaconda**, é um software que reúne várias bibliotecas python e que vai te ajudar a consumir os dados do conjunto utilizando o python.

Link para download => <https://www.anaconda.com/download>

Após efetuar o download, é só clicar em **NEXT** até concluir a instalação, uma vez finalizada a instalação procure na barra de pesquisa do seu computador por **Anaconda Navigator**. Você verá a tela abaixo, basta clicar em **launch**.

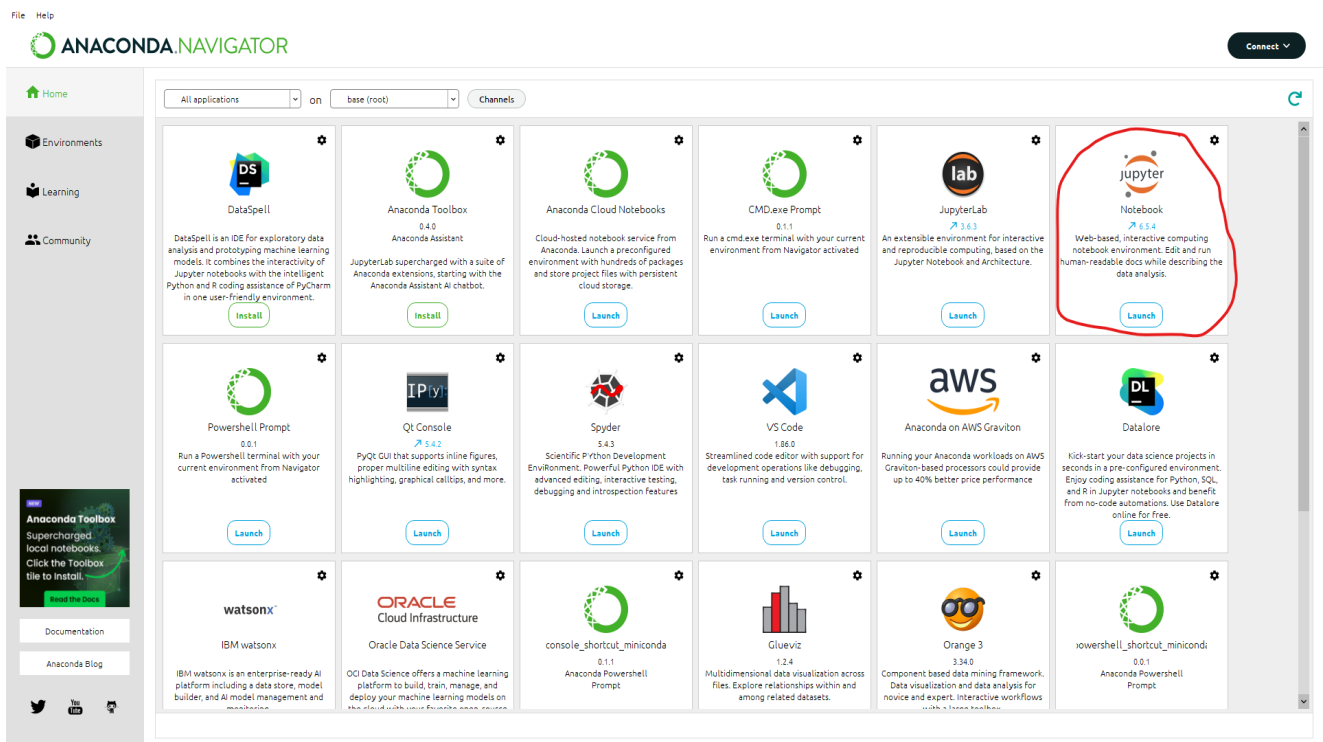


Figura 14

Será carregada a tela abaixo onde **recomendo que você crie uma pasta** para que todos os notebooks de código que você criar fiquem dentro dela para uma melhor organização, pode dar o nome que você desejar.



Figura 15

Uma vez a pasta criada, é só entrar nela e **criar seus notebooks**.

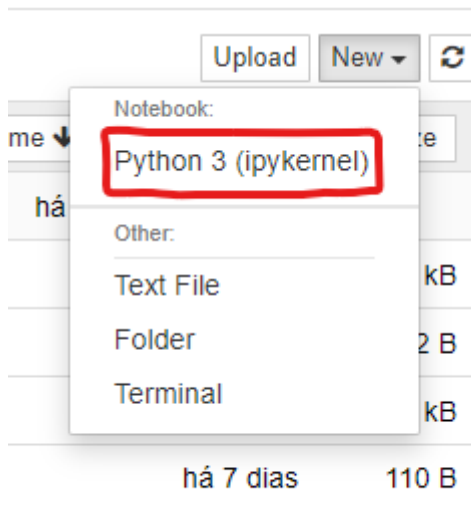


Figura 16

Clicando em **Python 3 (ipykernel)** será criado um notebook com o kernel do python onde você poderá desenvolver seus códigos. Feito isso, podemos passar para a parte do código para consumir os conjuntos.

Consumindo um conjunto que **não está** na camada cache.

Se o conjunto que você deseja consumir possui menos de 3 milhões de registros, ele **NÃO ESTÁ** na camada de cache. Então você usará o script python abaixo.

```
import requests

import csv

limit = 100000
total_offset = 9101629
current_offset = 0
```

Figura 17

Importe as duas bibliotecas **Requests** e **CSV**, caso deseje consumir os dados em **CSV**.

OBS: Se for usar os dados no formato Json, não precisa importar.

Para determinar o valor da variável **total_offset**, recomenda-se que utilize a API <https://www.bigdata.pe.gov.br/api/info> dentro do **Insomnia** para pegar a informação precisa da quantidade de registros.

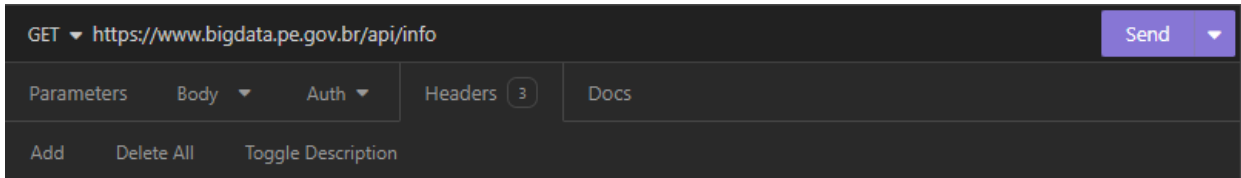


Figura 18

```
},
"qtd_registros": [
  [
    9101629
  ]
]
```

Figura 19

Logo em seguida, crie a variável **headers** com as seguintes informações:

```
: headers = {
  "Authorization": "",
  "limit": str(limit),
  "offset": "0",
  "Content-Type": "text/csv"
}
```

Figura 20

OBS: Caso queira consumir os dados no formato **JSON**, basta alterar a chave "Content-Type" para "application/json", ficando da seguinte forma: **"Content-Type": "application/json"**

O trecho abaixo corresponde a criação do arquivo e a interação para escrever os dados dentro dele. **Na função open(), você vai indicar o caminho onde você quer que salve o arquivo**, caso isso não seja informado, o arquivo será criado na mesma pasta em que o código está sendo executado.

Exemplo de caminho para salvar o arquivo: **"C:**

\Users\seuUser\Documents\nomedoarquivo.csv"

```

# Abre o arquivo fora do loop
with open('nome_do_arquivo.csv', 'w', encoding='utf-8') as json:
    while current_offset <= total_offset:
        # Atualiza o offset no cabeçalho da requisição
        headers["offset"] = str(current_offset)
        # Faz a requisição com o offset atual
        resp = requests.get('http://www.bigdata.pe.gov.br/api/buscar', headers=headers)
        # Processa os dados recebidos
        data = resp.text # Agora, data contém os dados no formato CSV
        # Escreve os dados no arquivo
        csv_file.write(data)
        print(f"Offset {current_offset}: {len(data)} registros")
        # Atualiza o offset para a próxima iteração
        current_offset += limit

    if total_offset > current_offset:

```

Figura 21

Para conjuntos muito grandes e que não estão na camada de cache, você pode **fracionar a extração** gerando múltiplos arquivos com lotes de registros. Isso vai facilitar até em caso de falha, você poderá continuar a extração de onde você parou.

Ao invés de usar o trecho de código da [Figura 21](#), você usará o trecho de código da imagem abaixo.

```

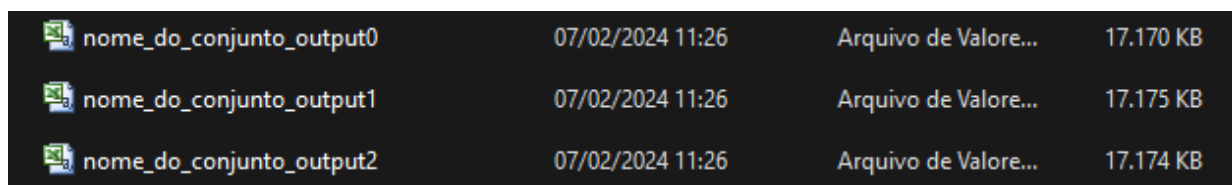
file_count=0
while current_offset < (total_offset - limit):

    response = fetch_data(current_offset)
    csv_name = 'output' + str(file_count)
    with open('nome_do_conjunto/'+csv_name+'.csv', 'w') as csvfile:
        csvfile.write(response)
    file_count += 1
    current_offset += limit
    print(file_count, current_offset)

```

Figura 22

A variável **csv_name** é dinâmica e vai sempre trocar o nome do arquivo, para o lote de registros que ela está carregando. Ex: Supondo que você ta extraindo a cada loop 100 mil registros. O nome do arquivo para a primeira interação ficaria: **nome_do_conjunto_output_0**. Para a Segunda interação: **nome_do_conjunto_output_1**. E assim sucessivamente. Lembrando que, cada output deste, possui a **quantidade de registros equivalente ao que você definiu no limit**.



nome_do_conjunto_output0	07/02/2024 11:26	Arquivo de Valore...	17.170 KB
nome_do_conjunto_output1	07/02/2024 11:26	Arquivo de Valore...	17.175 KB
nome_do_conjunto_output2	07/02/2024 11:26	Arquivo de Valore...	17.174 KB

Figura 23

No exemplo, cada conjunto desses possui 100 mil registros, que **foi o que eu defini na variável limit**.

Consumindo um conjunto que **está na camada** cache.

Conjuntos que estão na camada de cache, tem que ser extraído de uma forma diferente. E para isso foi mudado alguns detalhes no código colocarei prints das etapas do código abaixo.

OBS: Com o código abaixo, **sempre será gerado um arquivo CSV** no final. Apesar de ter uma parte no código sobre *JSON*, **no final sempre será gerado um arquivo CSV**.

Vamos começar importantando as bibliotecas que serão utilizadas ao longo do código.

```
import requests
import pandas as pd
import time
```

Figura 24

Agora iremos definir algumas variáveis. Não esqueça de colocar a **Hash do token** do conjunto como valor da variável **TOKEN**. É importante que essa hash seja colocada entre aspas simples. E na variável **CSV_FILENAME** é onde você vai colocar o **nome do arquivo**. É importante que no final do nome do arquivo, você deixe a extensão **.CSV** igual no exemplo.

```

print("INICIO SCRIPT")

token = ""
url = "https://www.bigdata.pe.gov.br/api/buscar"
headers = {
    "Content-Type": "application/json",
    "Authorization": token,
    "Limit": "10000",
    "Cache": "true"
}

pagina = 0
total = 0
start_time = time.time()
response = requests.get(url, headers=headers)
elapsed_time = time.time() - start_time
print("Request: pagina {} - {} s".format(round(pagina, 2), elapsed_time))

cursor = response.headers.get("Cursor-Proximo")

# Nome do arquivo CSV
csv_filename = "manual_teste_cache.csv"

```

Figura 25

Realizada as devidas alterações, **nada mais** precisará ser editado.

```

while True:
    loop_time = time.time()
    headers['Cursor'] = cursor
    response = requests.get(url, headers=headers)
    cursor = response.headers.get("Cursor-Proximo")
    loop_elapsed = time.time() - loop_time
    pagina += 1
    num_docs = len(response.json())
    total += num_docs
    print("Request: pagina {} | {} s | {} documentos".format(round(pagina), loop_elapsed, num_docs))

    if num_docs < 1:
        break

    # Criação do DataFrame do pandas automaticamente
    df = pd.json_normalize(response.json())

    # Se for a primeira página, salva os cabeçalhos
    if pagina == 1:
        df.to_csv(csv_filename, mode='w', header=True, index=False, encoding='utf-8')
    else:
        # Adiciona as linhas ao arquivo CSV
        df.to_csv(csv_filename, mode='a', header=False, index=False, encoding='utf-8')

print("Total documentos: {}".format(total))
print("FIM SCRIPT")

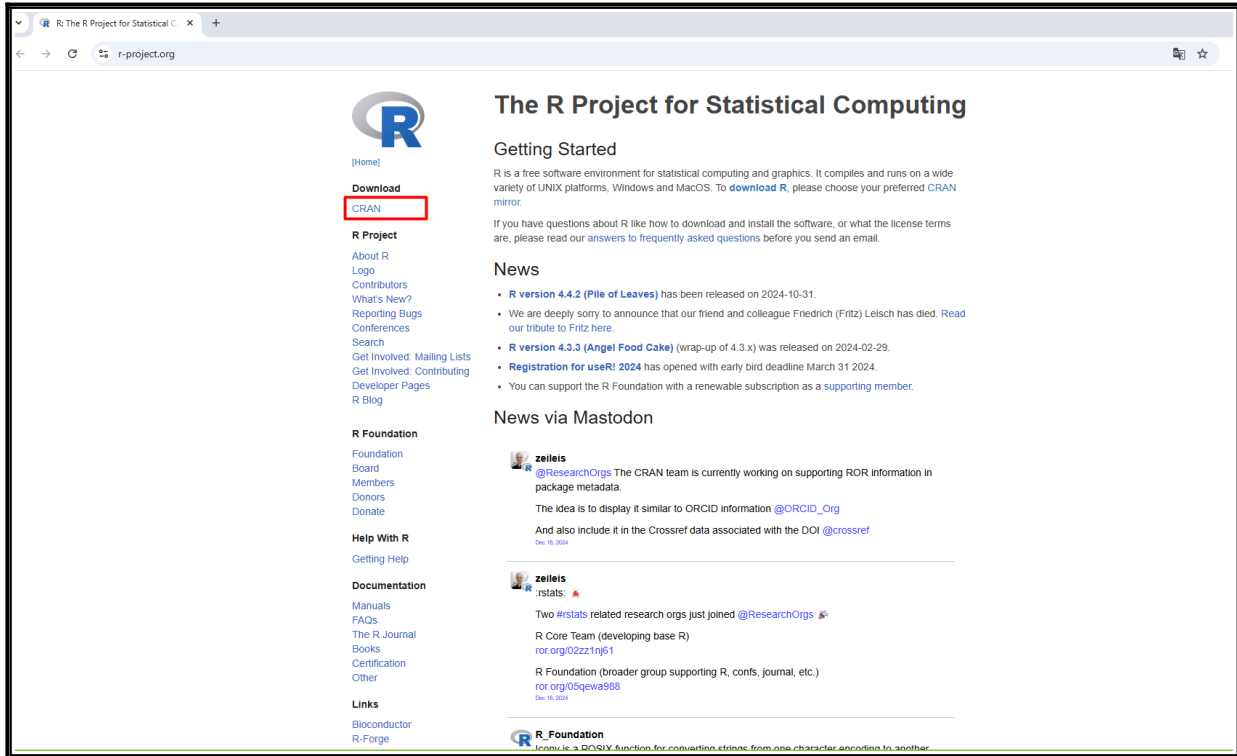
```

Figura 26

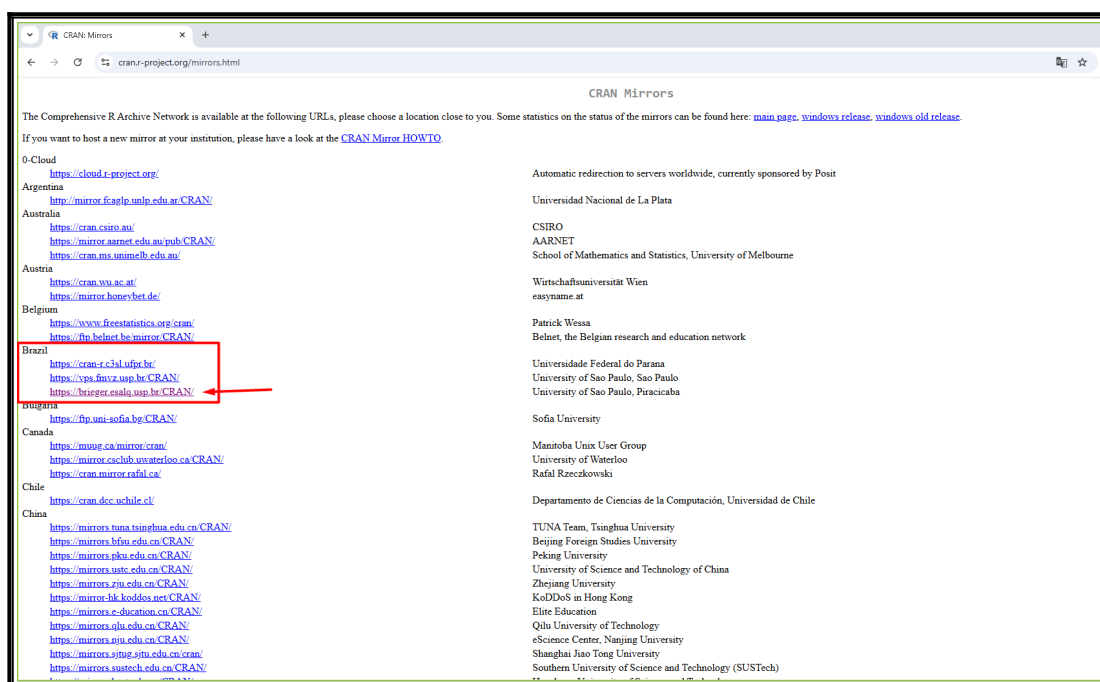
Recomendo que você adicione todo o código, **inclusive a parte do print("FIM SCRIPT")** para que você possa **ter a certeza** que a extração dos dados foi realizada com **sucesso**.

8. Tecnologia R Studio

1. Em seu navegador acesse o site: R: [The R Project for Statistical Computing \(r-project.org\)](https://r-project.org)
2. Clique no link CRAN no lado esquerdo da tela.



3. Escolha um dos portais de acesso de acordo com a sua preferência. **Observação:** O próprio site aconselha escolher o portal que faz referência a localidade mais próxima de você. Portanto escolha um dos que estão englobados no país em que você se encontra.



The Comprehensive R Archive Network

Download and Install R

Precompiled binary distributions of the base system and contributed packages. **Windows and Mac** users most likely want one of these versions of R:

- [Download R for Linux \(Debian, Fedora/Redhat, Ubuntu\)](#)
- [Download R for macOS](#)
- [Download R for Windows](#)

R is part of many Linux distributions, you should check with your Linux package management system in addition to the link above.

Source Code for all Platforms

Windows and Mac users most likely want to download the precompiled binaries listed in the upper box, not the source code. The sources have to be compiled before you can use them. If you do not know what this means, you probably do not want to do it!

- The latest release (2024-10-31, Pile of Leaves) [R4.4.2.tar.gz](#), read [what's new](#) in the latest version.
- The CRAN directory [src/base-prerelease](#) contains R alpha, beta, and rc releases as daily snapshots in time periods before a planned release.
- Between releases, the same directory [src/base-prerelease](#) contains snapshots of current patched and development versions. Please read about [new features and bug fixes](#) before filing corresponding feature requests or bug reports.
- Alternatively, daily snapshots are [available here](#).
- Source code of older versions of R is [available here](#).
- Contributed extension [packages](#).

Questions About R

- If you have questions about R like how to download and install the software, or what the license terms are, please read our [answers to frequently asked questions](#) before you send an email.

Supporting CRAN

- CRAN operations, most importantly hosting, checking, distributing, and archiving of R add-on packages for various platforms, crucially rely on technical, emotional, and financial support by the R community.

Please consider making [financial contributions](#) to the R Foundation for Statistical Computing.

5. Caso o seu computador tenha o Windows como o sistema operacional, clique em base e em seguida faça o download.

R for Windows

Subdirectories:

- [base](#)
- [contrib](#)
- [old.contrib](#)
- [Rtools](#)

Binaries for base distribution. This is what you want to [install R for the first time](#).

Binaries of contributed CRAN packages (for R >= 4.0.x).

Binaries of contributed CRAN packages for outdated versions of R (for R < 4.0.x).

Tools to build R and R packages. This is what you want to build your own packages on Windows, or to build R itself.

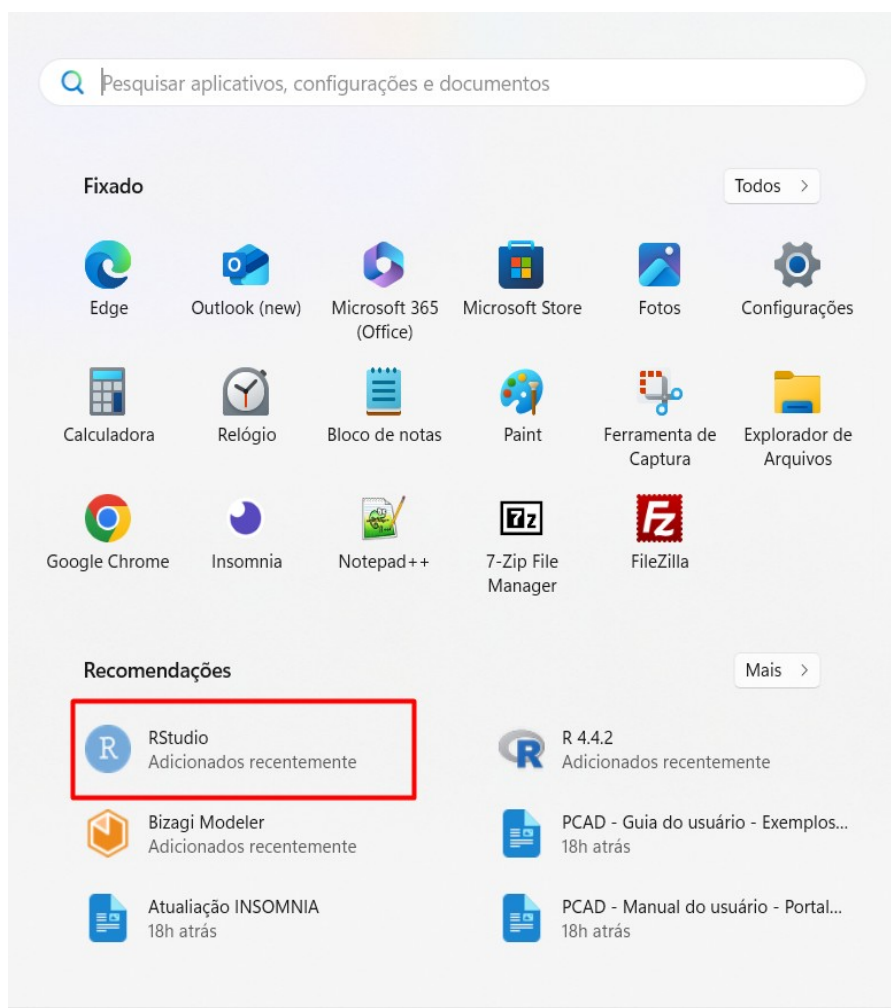
Please do not submit binaries to CRAN. Package developers might want to contact Uwe Ligges directly in case of questions / suggestions related to Windows binaries.

You may also want to read the [R FAQ](#) and [R for Windows FAQ](#).

Note: CRAN does some checks on these binaries for viruses, but cannot give guarantees. Use the normal precautions with downloaded executables.



6. Após o download ser realizado, clique duas vezes no arquivo e execute-o.
7. Depois da execução e a instalação ser finalizada, o R está pronto para uso.
8. DICA: Após a instalação do R geralmente um ícone do programa é criado na área de trabalho. Caso isso não ocorra é só ir no menu iniciar do computador e verificar na área “Programas”. Ou nos aplicativos **“recomendações”** recentemente.

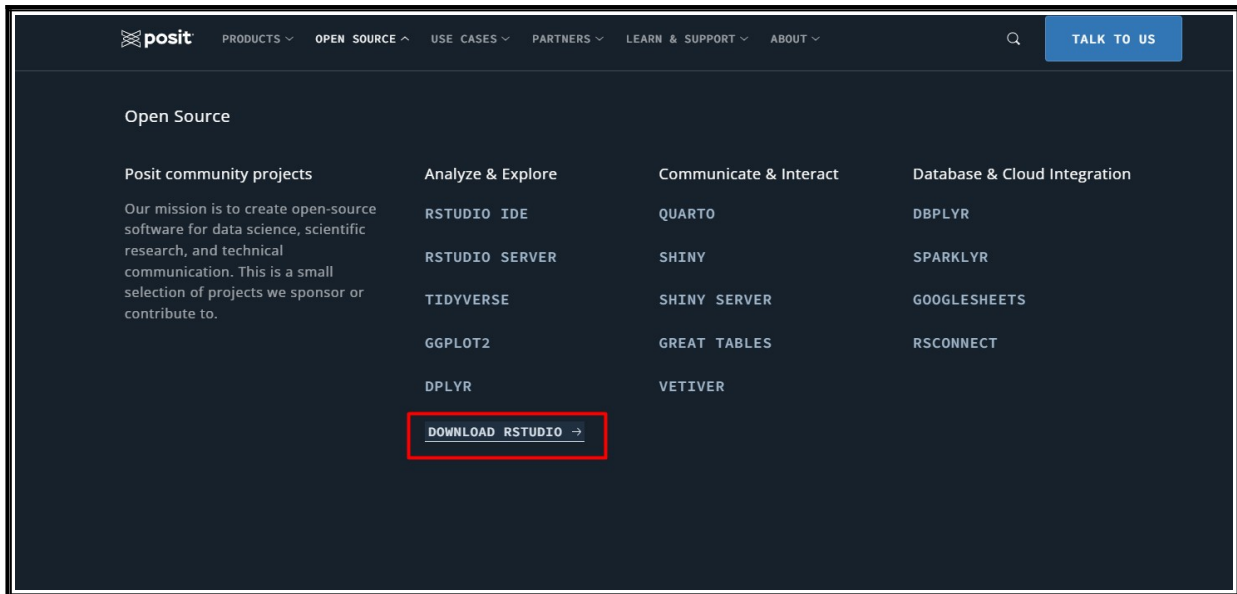


9. Agora iremos seguir para a instalação do Rstudio.

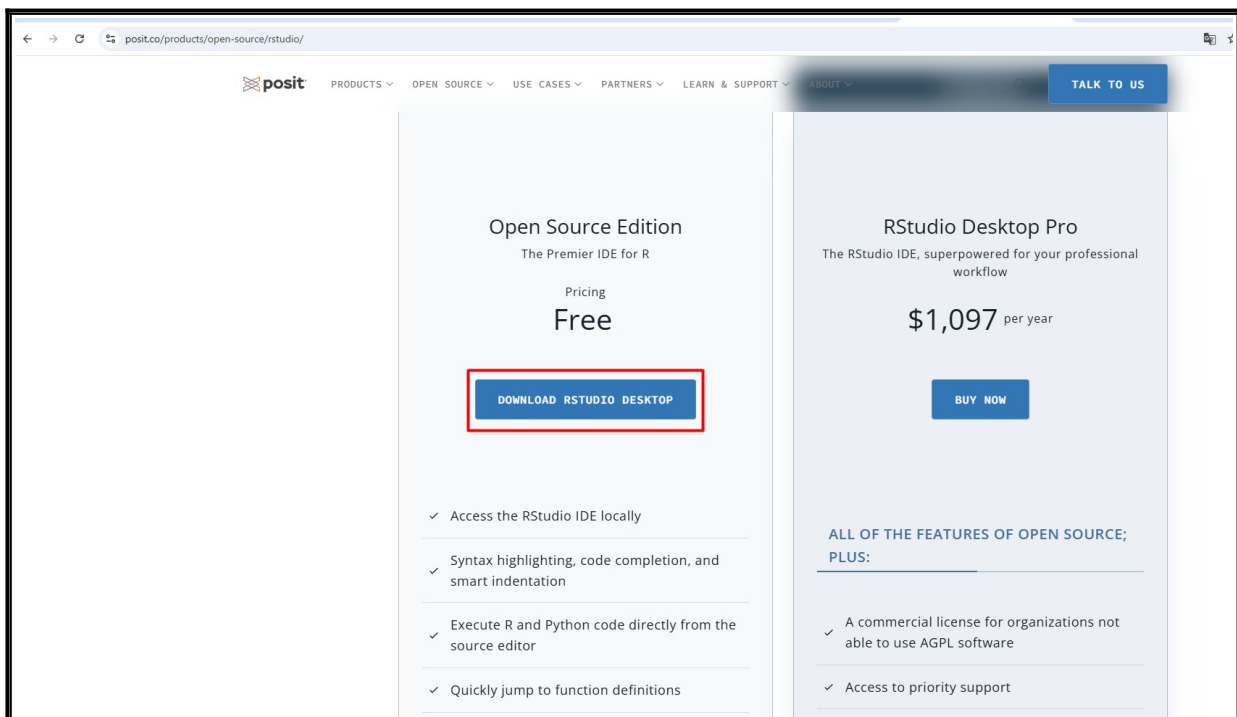
OBSERVAÇÃO: A explicação sobre a diferença entre os dois será explicada durante as aulas. Nesse tutorial iremos nos atentar somente às instalações. É imprescindível a instalação dos dois programas para as aulas.

RSTUDIO

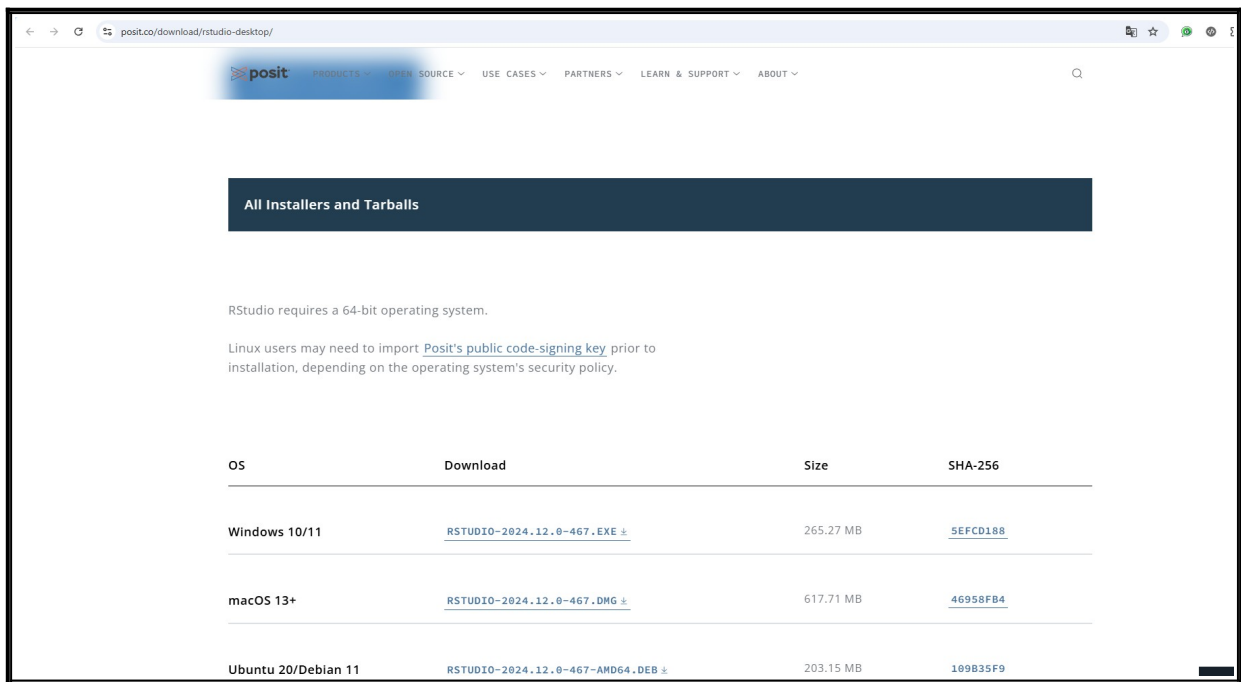
1. Acesse o site: [Posit | The Open-Source Data Science Company](https://posit.co) e clique em DOWNLOAD RSTUDIO.



2. Desça a página um pouco e clique em DOWNLOAD em RStudio Desktop



3. Na próxima página, caso o seu computador tenha o Windows como sistema operacional, clique no botão de download direto conforme mostrado na imagem. Caso contrário, procure o sistema operacional do seu computador nos links disponíveis na mesma página, logo abaixo na seção “All Installers and Tarballs”. A opção **“1: Install R”** é somente para quem ainda não instalou o [Programa R](#) que instalamos na primeira seção deste tutorial. Se você seguiu o passo a passo da primeira seção, não precisa se preocupar com a opção **“1: Install R”**.



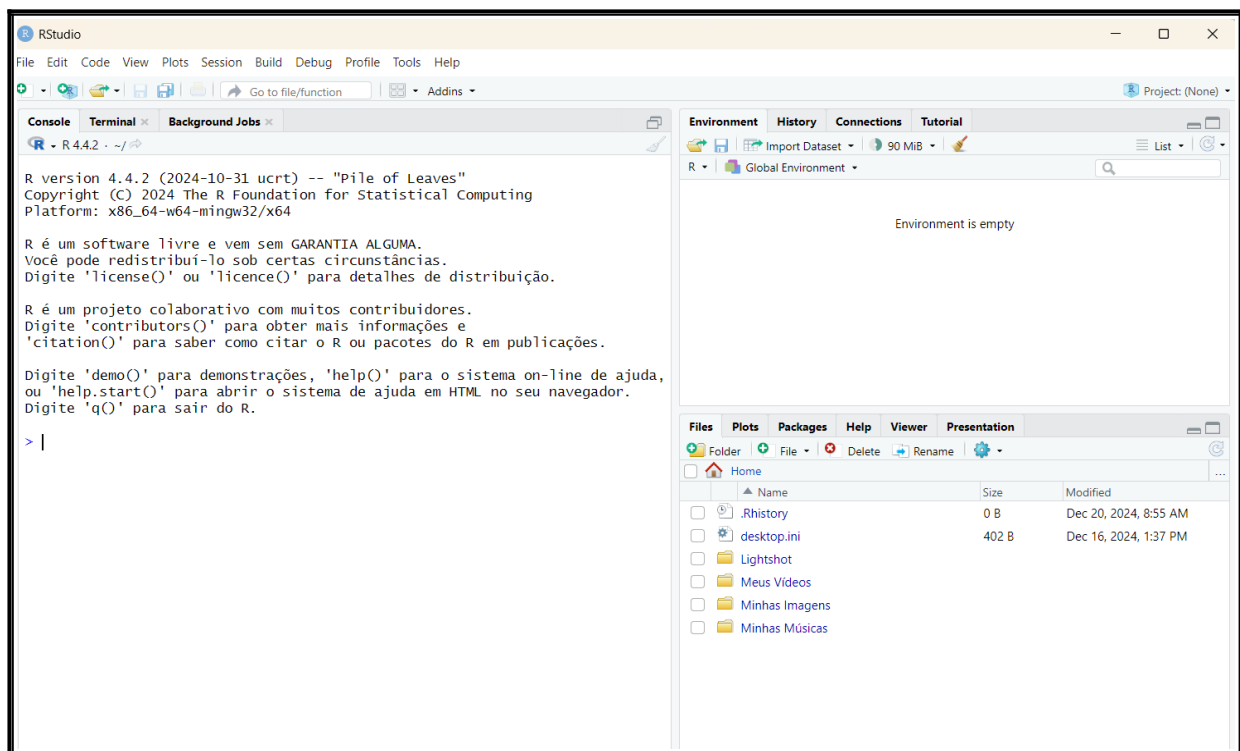
The screenshot shows the Posit website's download page for RStudio desktop. The page has a dark blue header with the Posit logo and navigation links. Below the header, there is a section titled "All Installers and Tarballs" in a dark blue box. The main content area contains text stating that RStudio requires a 64-bit operating system and that Linux users may need to import Posit's public code-signing key. Below this, there is a table with four columns: OS, Download, Size, and SHA-256. The table lists three operating systems: Windows 10/11, macOS 13+, and Ubuntu 20/Debian 11, each with a corresponding download link, size, and SHA-256 hash.

OS	Download	Size	SHA-256
Windows 10/11	RSTUDIO-2024.12.0-467.EXE ↕	265.27 MB	5EFC0188
macOS 13+	RSTUDIO-2024.12.0-467.DMG ↕	617.71 MB	46958FB4
Ubuntu 20/Debian 11	RSTUDIO-2024.12.0-467-AMD64.DEB ↕	203.15 MB	109B35F9

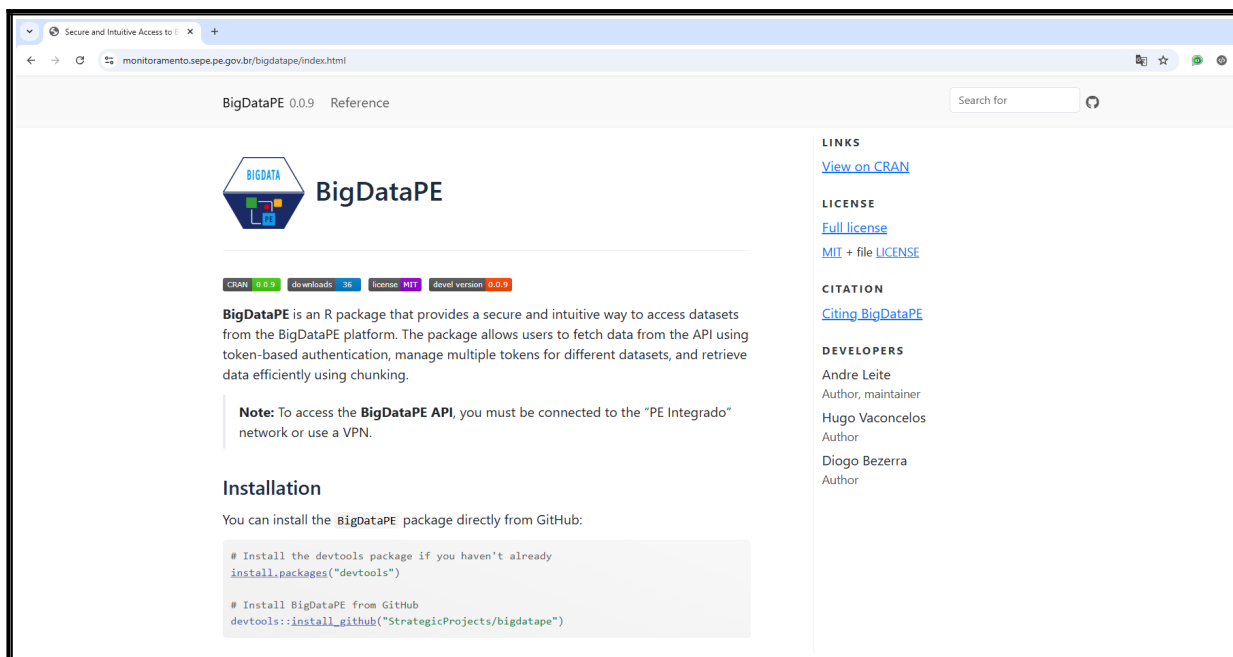
Abrir a aplicação



Tela Inicial do R Studio.



Seguir os passos de instalação de pacotes conforme [Pacote R Studio for Big Data PE](#)

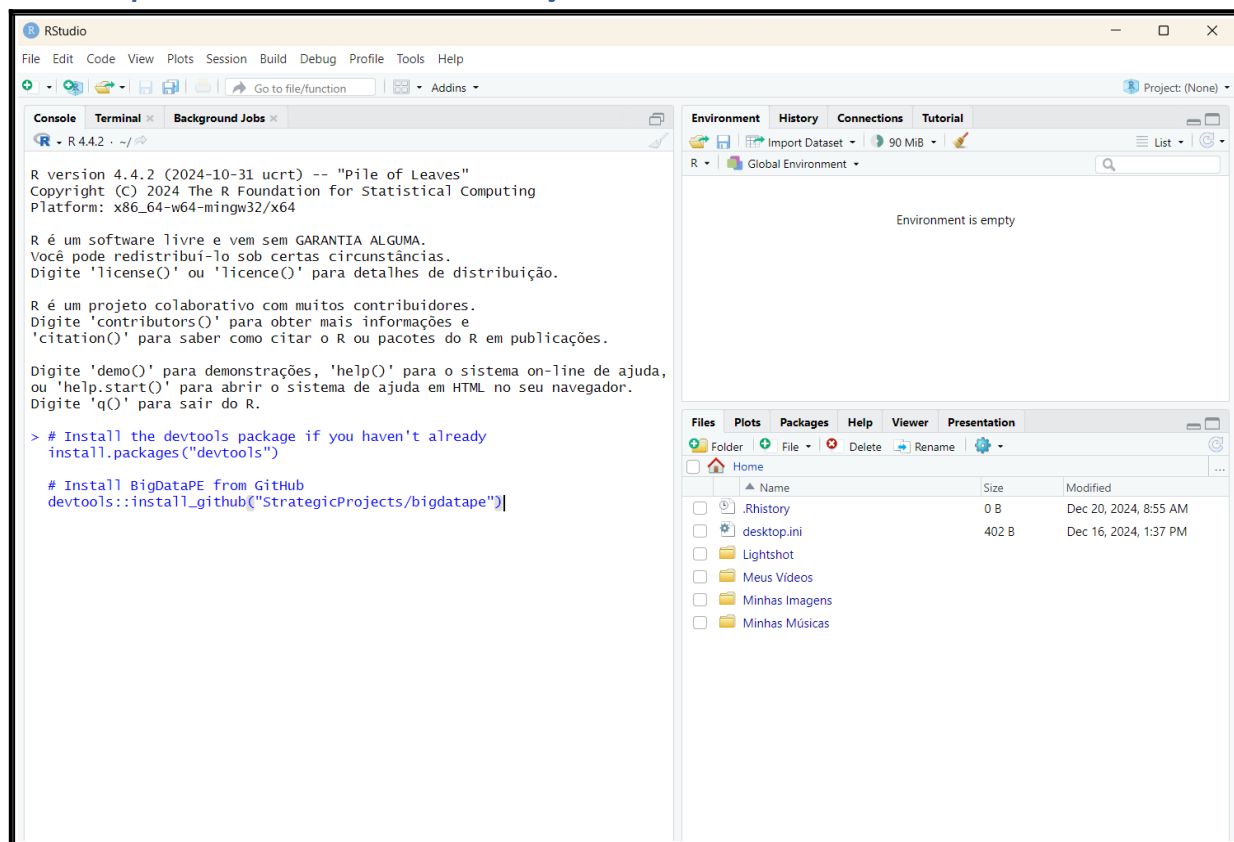


Pacote R Studio for Big Data PE

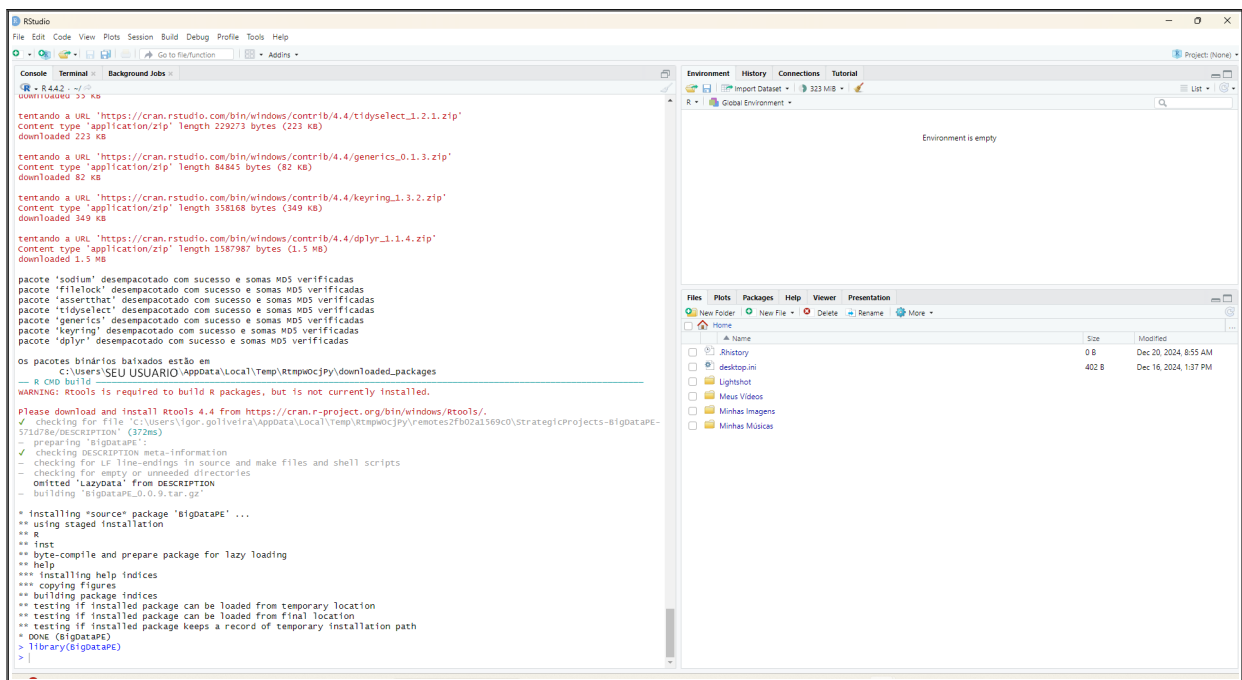
Incluir comando conforme link informado anteriormente.

Aguardar a instalação dos pacotes e dependências para utilizar o pacote no R Studio.

Comando para serem incluindo na instalação:



Confirmação de instalação do pacote:



Observação: Após as instalações e pacotes, seguir com o uso das funcionalidades conforme necessidade.

Leite A, Vaconcelos H, Bezerra D (2024). *BigDataPE: Secure and Intuitive Access to 'BigDataPE' 'API' Datasets*. R package version 0.0.9, <<https://github.com/StrategicProjects/bigdatape>>.